

Rewards System

Shopify +

1. What this application is

- Rewards System is a scoped loyalty (points) system for a Shopify store, built in PHP Laravel.
- Laravel stores customer points. Shopify stays the store where orders are paid and discount codes are used.
- This is not a full Smile.io / Yotpo clone. It covers the usual assessment slice: earn on paid orders, reverse on refunds, redeem for a single-use discount, HMAC webhooks, and Admin GraphQL.
- Shoppers use the Shop page. You use Admin to connect Shopify, set points rules, and manage rewards.

2. Do you need a GraphQL account?

- No. You do not create a GraphQL account, username, or subscription.
- GraphQL is the language Shopify uses for its Admin API. The app already contains the queries and mutations.
- The only credentials you need are from a Shopify custom app on YOUR store: Admin API access token (starts with shpat_) and the API secret key (webhook HMAC).
- When a customer redeems a reward, Laravel sends a GraphQL mutation named `discountCodeBasicCreate` to `https://YOUR-SHOP.myshopify.com/admin/api/2025-01/graphql.json` with header `X-Shopify-Access-Token`.
- When you click Test connection, Laravel sends a GraphQL query named `shop` to confirm the token works.
- When you click Turn on order webhooks, Laravel sends `webhookSubscriptionCreate` for `ORDERS_PAID` and `REFUNDS_CREATE`.
- If Live mode is off, the same GraphQL documents are logged locally (demo mode) so you can try the app without a token.

3. How the system works (end to end)

- Earn: Shopify sends a webhook when an order is paid. Laravel reads the order subtotal, multiplies by points per dollar, and writes an append-only ledger row. The same order cannot earn twice (idempotent key `shopify:order:{id}:earn`).
- Refund: Shopify sends `refunds/create`. Laravel reverses points (never below zero).
- Redeem: Shopper has enough points. Laravel calls Shopify GraphQL to create a one-time discount code, then deducts points and shows the code (RWD-XXXXXXX). The shopper enters that code at Shopify checkout.
- Admin adjustment: You can add or remove points for goodwill.

- Customer email is the join key between Shopify orders and the rewards card on /.

4. Put the app on a live server

- Requirements: PHP 8.3+, Composer, SQLite (or MySQL), HTTPS domain pointed at the server.
- Upload the project (git clone), then:
 - composer install --no-dev --optimize-autoloader
 - cp .env.example .env && php artisan key:generate
- Set in .env: APP_ENV=production, APP_DEBUG=false, APP_URL=https://your-domain.com, SHOPIFY MOCK=false
- touch database/database.sqlite && php artisan migrate --force && php artisan db:seed --force (seed is optional on production if you do not want demo customers).
- chmod -R ug+rw storage bootstrap/cache
- Point the web root at the public/ folder (Apache DocumentRoot or Nginx root). Do not expose .env.
- Confirm https://your-domain.com/up returns OK (Laravel health check).
- Open https://your-domain.com/admin/settings and connect Shopify using the public HTTPS URL of this server (not localhost).

5. Connect your live Shopify store (easy path)

- In Shopify Admin: Settings -> Apps and sales channels -> Develop apps -> Allow custom app development (if asked) -> Create an app. Name it Rewards.
- Configuration -> Admin API integration -> Edit. Enable: read_orders, read_customers, write_discounts. Save.
- Install app. Reveal Admin API access token once and copy it (shpat_...). Copy API secret key as well.
- Shop domain must be your-store.myshopify.com (Settings -> Domains shows it). A custom domain like shop.com is not the API hostname.
- In this app: Admin -> Shopify.
- Paste shop domain, access token, API secret, and Public HTTPS URL = https://your-domain.com (no /admin).
- Check Use the live Shopify API. Click Save.
- Click Test connection. You should see your real shop name.
- Click Turn on order webhooks. Shopify will POST to https://your-domain.com/api/webhooks/shopify
- Place a small paid test order on the live store. Open Admin -> Customers. The buyer email should have points.
- On Shop (/), enter that email -> Redeem. Copy the discount code into Shopify checkout.

6. Daily use - shoppers

- Go to the Shop page (/).
- Enter the same email used at Shopify checkout. Click Show my points.
- Buy and earn on the demo products only simulates an order. Real points on a live store come from real paid orders.
- When you have enough points, click Redeem. Use the code at checkout on the Shopify store.

7. Daily use - merchant admin

- Home: totals and recent activity. You can apply a test purchase or refund before Shopify is connected.
- Rewards: create dollar-off or percent-off rewards and how many points they cost.
- Customers: balances, history, bonus points.
- Shopify: connection, earn rules (points per dollar, minimum order, code expiry).

8. API endpoints (for a theme snippet, optional)

- GET /api/rewards/customer?email=customer@shop.com - balance, history, catalog.
- POST /api/rewards/redeem with email and reward_id - issues a discount code.
- POST /api/webhooks/shopify - Shopify only. Header X-Shopify-Topic is orders/paid or refunds/create. Header X-Shopify-Hmac-Sha256 must match the API secret.
- Protect the customer JSON API in production (do not leave it fully public if the site is on the internet). Add authentication before going fully public if you embed it in a theme.

9. Troubleshooting

- Test connection fails: wrong shop domain, token not installed, or missing scopes.
- Webhooks not registering: URL must be public HTTPS. Localhost and http:// are rejected.
- Orders paid but no points: check Webhook log on Shopify page; confirm HMAC secret; confirm order email is present; confirm financial_status is paid.
- Redeem stays in demo mode: check Use the live Shopify API and that a token is saved.
- Discount code not working on the store: confirm write_discounts scope and that you applied the code on the same shop the token belongs to.
- Git push to GitHub asks for a password: GitHub does not accept account passwords. Use a personal access token (repo scope) as the password, or SSH keys.

10. Security

- Never commit .env or the shpat_ token.
- On a live server keep APP_DEBUG=false.

- Live mode requires the webhook secret. Unsigned webhooks are rejected.
- Put Admin behind login before you expose the site to the public internet (this assessment slice ships Admin open for demo).